DUALSERVE Complete System Architecture & Walkthrough

DUALSERVE is a state-of-the-art, multi-role hybrid service marketplace designed for **Household and Towing Services**. Built on top of a highly responsive, high-performance architecture, the system leverages a **Flutter multi-platform application** as the frontend, and a serverless **Firebase backend** as the core engine.

This walkthrough outlines the complete design, workflows, services, functions, and models that compose the entire DUALSERVE ecosystem.

1. System Overview & Technology Stack

DUALSERVE uses a state-of-the-art serverless architecture to guarantee real-time reactivity, high availability, offline availability, and military-grade transactional consistency.

```
graph TD
  A["Flutter Multi-Platform Frontend (Web / Android / iOS)"] -->|Authentication| B["Firebase Authentication"]
  A -->|Real-time Data Streams| C["Cloud Firestore (with Offline Cache)"]
  A -->|Image Assets / Documents| D["Cloud Storage"]
  A -->|FCM Push Notifications| E["Firebase Cloud Messaging"]
  A -->|Callable / HTTPS APIs| F["Firebase Cloud Functions"]
  F -->|SMTP Transport| G["Nodemailer (Credentials Setup)"]
  F -->|Maps APIs| H["Google Places API"]
  F -->|Scheduled Events| I["Cloud Scheduler / PubSub"]
  I -->|Trigger Actions| F
```

1.1 Core Frontend Tech Stack

- **Framework:** Flutter (Dart) for high-fidelity Android, iOS, and PWA compilation.
- **State Management:** `Provider` coupled with `ChangeNotifier` for reactive,

2. Core Workflows & Lifecycles

The core business flow moves from a customer request (Booking) to administrator planning (Task with Assets) and provider work execution (Task In Progress), culminating in final payment processing (Transaction).

```
sequenceDiagram
  autonumber
  actor Customer
  actor Provider
  actor Admin

  Customer->>Customer: Selects Location & Service
  Customer->>Customer: Estimates Cost (Base Price + Distance Surcharge)
  Customer->>BookingService: Create Booking (Locks time slot atomically)
  BookingService-->>Admin: New Booking Notification (FCM Trigger)

  Note over Admin: Option A: Booking accepted by Provider
  Provider->>BookingService: Accept Booking (Firestore Transaction)
  BookingService->>TaskService: Auto-Generate Task (Status: ASSIGNED)

  Note over Admin: Option B: Manual Task Creation
  Admin->>TaskService: Manual Task Creation (Status: UNASSIGNED)
  Admin->>TaskService: Assign Task to Provider (Status: ASSIGNED)

  Provider->>AssetService: Assign Truck & Gear to Task (Log Resource Usage)
  Provider->>TaskService: Start Task (Status: IN_PROGRESS)
  Provider->>TaskService: Complete Milestone (Updates progress %)
```

3. Data Dictionary (Collections & Firestore Schema)

3.1 Users (`users/{uid}`)

Contains primary user profile cards, defining credentials, role levels, and basic profile configuration.

Field	Type	Description
`uid`	String	Unique Authentication ID.
`name`	String	User's full name.
`email`	String	Primary email address (lowercased).
`phone`	String	Contact number.
`role`	String	User role: `customer`, `provider`, `admin`, `pending_provider`.

4. Firebase Cloud Functions (`functions/index.js`)

DUALSERVE's Cloud Functions manage critical operations, background calculations, and sensitive actions that must never be exposed to client-side manipulation.

4.1 Custom Claims Provisioning (`setAdminRole`)

- **Purpose:** Ensures high-security boundary controls.
- **Trigger:** HTTPS Callable.
- **Process:**
    i. Validates that the executing user contains the administrative claim (`context.auth.token.admin === true`).
    ii. Validates existence of targeted user using `admin.auth().getUser(targetUid)`.
    iii. Provisions custom claims payload `{ admin: true }` to Firebase Auth token metadata.

5. Flutter Core Services Architecture (`lib/services/`)

5.1 `BookingService` (Booking Management)

Manages reservations, slot locks, and transitions from booking requests to tasks.

```
// Key Methods:
Future<String> createBooking(Booking booking);
Future<void> acceptBooking(String bookingId, String providerId);
Future<void> rejectBooking(String bookingId);
Future<void> cancelBooking(String bookingId);
Stream<List<Booking>> getCustomerBookings(String customerId);
Stream<List<Booking>> getProviderBookings(String providerId);
```

- **Atomic Time Slot Locking:** Uses document transactions to create a unique slot reservation: `provider_slots/{providerId}_{dateISO}_{timeSlot}`. This completely blocks double bookings at the database level.
- **Booking Acceptance Transaction:** Accepts bookings inside an atomic Firestore

6. Frontend State Management (`UserProvider`)

State management uses Flutter's `Provider` and `ChangeNotifier` to manage user sessions and themes across the application.

```
class UserProvider with ChangeNotifier {
  Map<String, dynamic>? _userProfile;     // Base user record
  Map<String, dynamic>? _providerProfile; // Provider metadata (schedule, rating)
  String? _role;                          // Role label (admin, customer, etc.)
  bool _isDarkMode = false;               // Local theme choice
  bool _isLoading = false;                // Activity indicator flag

  // Getters, theme triggers, and async loading commands...
}
```

6.1 Authentication Bootstrapping Flow

On application boot, the root MaterialApp mounts an `AuthWrapper` connected to FirebaseAuth's auth state stream:

7. Complete UI Architecture Map

7.1 Customer Space (`lib/screens/customer/`)

- `CustomerMainLayout`: Contains bottom navigation controls routing to active views.
- `CustomerHome`: Offers a clean, grid-based dashboard for quick access to booking forms.
- `BookingScreen`: Includes booking selection forms with distance calculations and real-time pricing estimates.
- `CustomerTrackingScreen` / `ServiceTracking`: Displays active services, current milestones, and live map tracking.
- `BillingHistoryScreen`: Shows invoice lists, cost breakdowns, and receipts.

7.2 Provider Space (`lib/screens/provider/`)

- `ProviderMainLayout`: Provides navigation tools for active duties, schedules, and